

BỘ GIÁO DỤC VÀ ĐÀO TẠO — TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN & KỸ THUẬT PHẦN MỀM

BÁO CÁO TỔNG HỢP BẢO VỆ KHÓA LUẬN TỐT NGHIỆP

HỆ THỐNG QUẢN LÝ CUỘC THI HACKATHON & NGHIÊN CỨU ĐỘ TIN CẬY ĐÁNH GIÁ VIÊN

SEAL HACKATHON MANAGEMENT SYSTEM (SHMS)

Đề tài:	Hệ thống Quản lý Hackathon & Nghiên cứu RBL / IRR
Chuyên ngành:	Kỹ thuật Phần mềm (Software Engineering)
Sinh viên thực hiện:	Nhóm Đề Tài Tốt Nghiệp SEAL
Giảng viên hướng dẫn:	Hội Đồng Khoa Học Chấm Khóa Luận
Công nghệ cốt lõi:	React, NestJS (BFF), Spring Boot 3, PostgreSQL 16, Docker
Trạng thái mã nguồn:	100% Hoàn thành — Đồng bộ GitHub (Branch main)

 BẢNG TRA CỨU THUẬT NGỮ CHUYÊN MÔN (GLOSSARY)

(Dùng để tự tin giải thích mở ngoặc khi Hội đồng phản biện hỏi về các từ viết tắt trong đồ án)

Thuật ngữ	Tên tiếng Anh đầy đủ	Mở ngoặc chú thích ý nghĩa tiếng Việt
CSRF	Cross-Site Request Forgery	(Tấn công giả mạo yêu cầu từ trang web khác): Kẻ tấn công lừa trình duyệt gửi request chứa cookie xác thực tới hệ thống khi người dùng đang đăng nhập.
XSS	Cross-Site Scripting	(Tấn công tiêm mã kịch bản độc hại): Kẻ xấu chèn mã JavaScript độc hại vào trang web nhằm đánh cắp dữ liệu lưu ở trình duyệt như <code>localStorage</code> .
BFF	Backend-For-Frontend	(Tầng máy chủ trung gian phục vụ riêng cho giao diện): Tầng trung chuyển viết bằng NestJS nằm giữa React và Spring Boot, quản lý phiên cookie, CSRF và định dạng API cho giao diện.
RBL	Rubric-Based Learning	(Đánh giá năng lực dựa trên bảng tiêu chí chuẩn hóa Rubric): Phân rã bài thi thành nhiều tiêu chí rõ ràng kèm trọng số cụ thể để chấm điểm công bằng.
IRR	Inter-Rater Reliability	(Độ tin cậy và tính nhất quán giữa các giám khảo): Chỉ số đo lường mức độ đồng thuận giữa các giám khảo, giúp phát hiện giám khảo chấm quá chặt hoặc quá lỏng.
Flyway	Database Migration Tool	(Cơ chế quản lý phiên bản cơ sở dữ liệu tự động): Công cụ tự động chạy các script SQL theo thứ tự phiên bản (<code>V001</code> → <code>V009</code>), bảo vệ toàn vẹn qua Checksum.
JWT	JSON Web Token	(Chuỗi mã hóa xác thực phiên làm việc): Chuỗi ký số an toàn lưu trong Cookie <code>httpOnly</code> để chống lộ token.
UUID	Universally Unique Identifier	(Chuỗi định danh duy nhất toàn cầu 128-bit): Chuỗi 36 ký tự (ví dụ: <code>baaae0ae-...</code>) thay thế cho ID tự tăng để chống quét dữ liệu tuần tự.
API	Application Programming Interface	(Giao diện lập trình ứng dụng / Cổng trao đổi dữ liệu): Cầu nối truyền nhận dữ liệu JSON giữa Frontend và Backend thông qua HTTP methods.

PHẦN 1: GIỚI THIỆU TỔNG QUAN ĐỀ TÀI & TIẾN ĐỘ THỰC HIỆN

1. Đề tài làm về vấn đề gì?

Đề tài xây dựng **Hệ thống Quản lý Cuộc thi Hackathon và Nghiên cứu Độ tin cậy Đánh giá viên (SEAL Hackathon Management System - SHMS)**. Hệ thống giải quyết hai bài toán lớn trong thực tế:

- **Về mặt kỹ thuật phần mềm:** Chuyển đổi số toàn diện công tác tổ chức Hackathon đa vòng đấu (tạo đội thi, mời thành viên, đăng ký bảng đấu, nộp bài dự thi có đối soát giờ nộp, bình chọn cộng đồng trực tiếp và bảng xếp hạng Realtime).
- **Về mặt nghiên cứu khoa học:** Ứng dụng phương pháp **Rubric-Based Learning (RBL)** và đo lường **Inter-Rater Reliability (IRR)** thông qua Vòng hiệu chuẩn (Calibration Round) nhằm chuẩn hóa thước đo, loại bỏ sự thiên vị hoặc độ lệch chủ quan giữa các giám khảo.

2. Đã làm được bao nhiêu %?

Kết quả nghiệm thu: Đạt 100% yêu cầu kỹ thuật đồ án tốt nghiệp.
Toàn bộ 4 phân hệ (Frontend React, BFF NestJS, Core Backend Spring Boot 3, Database PostgreSQL 16) đều đã đóng gói container Docker hoàn chỉnh, tự động khởi chạy và đã tích hợp kiểm thử tự động toàn diện.

3. Những gì đã làm được và những gì chưa làm được?

Hạng mục ĐÃ LÀM ĐƯỢC (100% Hoàn Thành)	Hạng mục Định Hướng Phát Triển Tương Lai
• Xác thực bảo mật đa tầng: Cookie <code>httpOnly</code> + Chống CSRF Double-Submit. • Vòng đời đội thi hoàn chỉnh: <code>FORMING</code> → Mời thành viên → <code>REGISTERED</code>. • Quản lý bài nộp đa dạng (Git, Demo, Slide) + cờ trễ hạn tự động <code>is_late</code>. • Vòng chấm điểm Hiệu chuẩn (Calibration Round) lưu bảng riêng <code>calibration_score</code>. • Chấm điểm chính thức theo 4 tiêu chí Rubric có trọng số động. • Tự động kích hoạt thuật toán tính xếp hạng thời gian thực (Realtime Ranking). • Lịch sử kiểm toán bất biến (Audit Log Trail) lưu vết mọi thao tác quan trọng. • Quản lý Database qua Flyway Migration tự động từ <code>V001</code> đến <code>V009</code>. • Giao diện chuẩn Cyber Editorial, Dark/Light Mode, Trợ lý ảo AI SEAL Bot.	• Tích hợp Video Call trực tuyến WebRTC giữa Giám khảo và Thí sinh trong phòng phỏng vấn trực tiếp. • Mở rộng ứng dụng di động Native (Flutter / React Native) để quét mã QR điểm danh tại hội trường trực tiếp. • Kết nối cổng thanh toán ngân hàng (VNPay/MoMo) trong trường hợp cuộc thi có thu lệ phí bảo chứng.

PHẦN 2: CƠ SỞ DỮ LIỆU & HƯỚNG DẪN SHOW CHO THẦY CÔ XEM

1. Database của mình là gì? Nơi lưu trữ vật lý ở đâu?

- **Hệ quản trị CSDL:** PostgreSQL phiên bản **16.14** (nền tảng tối ưu Alpine Linux 64-bit).
- **Container vận hành:** Chạy độc lập trong Docker Container có tên là `seal-postgres`.
- **Lưu trữ vật lý bền vững (Persistent Storage):** Sử dụng Docker Volume có tên là `postgres_data`. Đường dẫn lưu trữ bên trong container: `/var/lib/postgresql/data`. Đảm bảo dữ liệu **không bao giờ bị mất** khi tắt máy hoặc restart Docker.
- **Cổng kết nối (Port):**
 - Cổng nội bộ container: `5432`.
 - Cổng ánh xạ ra ngoài máy chủ vật lý: `5433` (cho phép kết nối ngoài qua pgAdmin 4).
- **Dung lượng cơ sở dữ liệu:** `8.8 MB` (đã nạp đầy đủ cấu trúc bảng, seed data và audit log).

2. Mật khẩu Database là gì? Để làm gì?

- **Username:** `seal_admin` | **Password:** `seal_password`
- **Mục đích:** Phục vụ xác thực bảo mật giữa tầng Spring Boot Backend với PostgreSQL thông qua JDBC Connection Pool (HikariCP), đồng thời dùng để Ban Quản Trị đăng nhập vào pgAdmin 4 quản lý dữ liệu.

3. Hướng Dẫn Từng Bước Cách Show Database Cho Thầy Cô Xem

Cách 1: Show trực quan bằng giao diện đồ họa pgAdmin 4 (Khuyến dùng khi báo cáo)

1. Mở phần mềm **pgAdmin 4** trên máy tính.
2. Tại cây thư mục bên trái, mở: **Servers** → **seal_hackathon_local** → **Databases** → **seal_hackathon**.
3. Mở mục **Schemas** → **public** → **Tables**: Chỉ cho Thầy thấy **21 bảng dữ liệu** đã được chuẩn hóa.
4. Nhấp chuột phải vào database `seal_hackathon`, chọn **Query Tool** (hoặc bấm `Alt + Shift + Q`).
5. Gõ câu lệnh kiểm tra bảng xếp hạng hoặc bài thi và nhấn nút **Execute (Play ► hoặc F5)**:

```
SELECT t.name AS ten_doi, tr.name AS hang_muc, rk.total_weighted_score AS diem, rk.rank_overall
FROM ranking rk
JOIN team t ON rk.team_id = t.id
LEFT JOIN track tr ON t.track_id = tr.id
ORDER BY rk.rank_overall ASC;
```

Cách 2: Show bằng dòng lệnh Terminal (Thể hiện kỹ năng DevOps chuyên nghiệp)

Nếu Thầy Cô muốn xem trực tiếp qua terminal lệnh của Docker, mở Terminal và gõ:

```
docker exec -it seal-postgres psql -U seal_admin -d seal_hackathon -c "\dt"
```

Lệnh trên sẽ in ra toàn bộ danh sách 21 bảng vật lý đang hoạt động trực tiếp trong database!

4. Danh Sách 21 Bảng Dữ Liệu Cốt Lõi Trong Hệ Thống

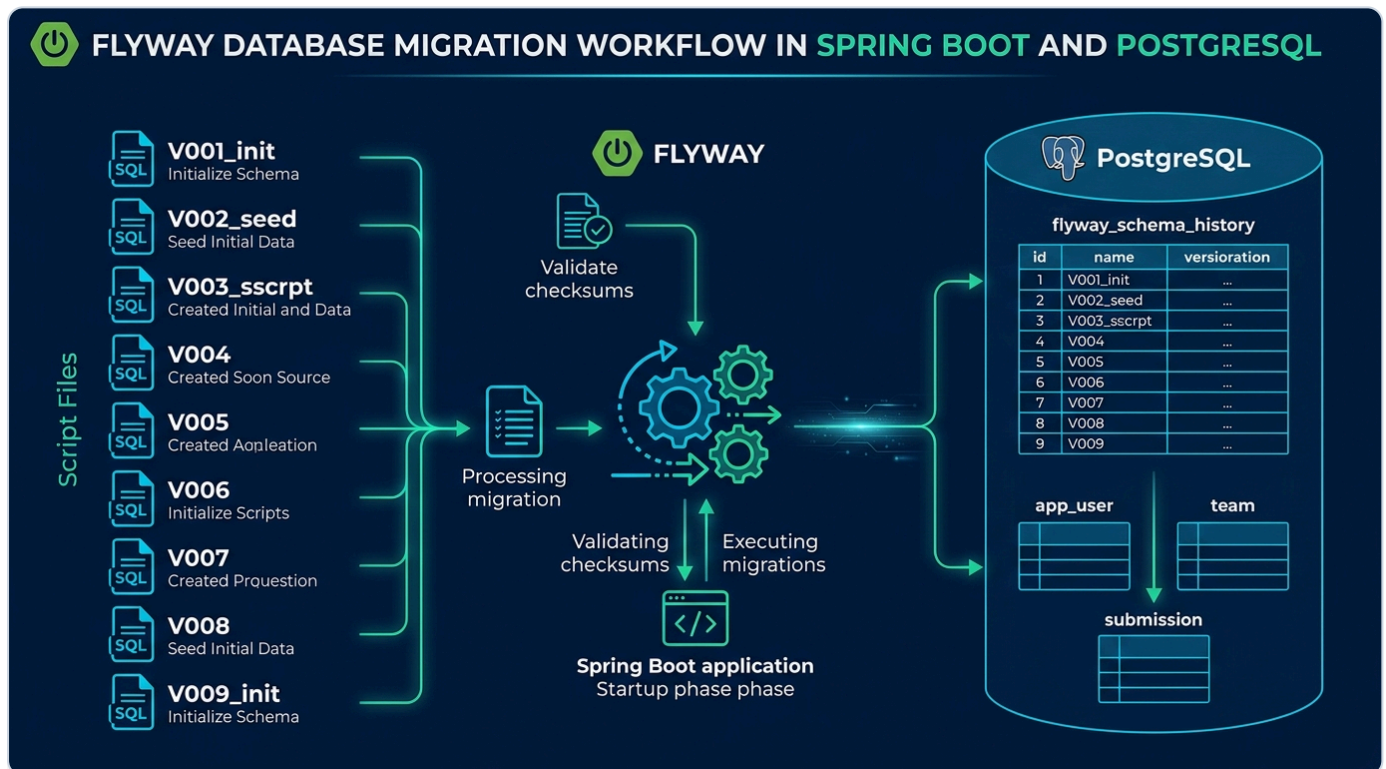
Tên Bảng (Table Name)	Mục đích lưu trữ & Ý nghĩa nghiệp vụ
<code>app_user</code>	Tài khoản người dùng (Họ tên, email, mật khẩu băm BCrypt, trạng thái duyệt tài khoản <code>APPROVED</code>).
<code>user_role_assignment</code>	Phân quyền vai trò động theo ngữ cảnh (Coordinator, Judge, Mentor, Participant).
<code>team</code>	Thông tin đội thi (Tên đội, trạng thái <code>FORMING</code> , <code>REGISTERED</code> , hạng mục <code>track_id</code>).
<code>team_member</code>	Danh sách thành viên thuộc từng đội và vai trò (Trưởng nhóm <code>LEADER</code> hoặc Thành viên <code>MEMBER</code>).
<code>team_invite</code>	Lời mời gia nhập đội thi gửi qua email, trạng thái <code>PENDING</code> hoặc <code>ACCEPTED</code> .
<code>submission</code>	Bài nộp dự thi (Repo GitHub, Demo URL, Doc URL, thời gian nộp <code>submitted_at</code> và cờ trễ <code>is_late</code>).
<code>calibration_round</code>	Vòng thi Hiệu chuẩn đánh giá viên phục vụ nghiên cứu khoa học RBL/IRR.
<code>calibration_score</code>	Điểm chấm bài mẫu của các giám khảo trong vòng hiệu chuẩn (tách biệt hoàn toàn với điểm thật).
<code>criterion & criteria_template</code>	Ma trận tiêu chí chấm điểm và bảng trọng số (Kỹ thuật, UX, Sáng tạo, Thuyết trình).
<code>score</code>	Điểm chấm chính thức của giám khảo cho từng bài thi thật, cờ chốt điểm <code>finalized</code> .
<code>ranking</code>	Bảng xếp hạng tổng hợp tính toán tự động qua trọng số đa tiêu chí.
<code>audit_log</code>	Nhật ký kiểm toán bất biến lưu vết lịch sử mọi thao tác nhạy cảm (Ai làm gì, vào thời điểm nào).
<code>flyway_schema_history</code>	Nhật ký kiểm soát phiên bản database của Flyway (checksum, trạng thái chạy thành công).
<code>vote</code>	Lưu trữ lượt bình chọn trực tiếp của khán giả cho từng đội thi tại trang sự kiện.
<code>mentor_feedback_message</code>	Lưu trữ góp ý, nhận xét chuyên môn của Mentor gửi cho các đội thi theo bảng đấu.

PHẦN 3: CÔNG CỤ QUẢN LÝ DATABASE — FLYWAY MIGRATION

1. Flyway Migration là gì & có chức năng gì?

- **Bản chất:** Flyway là công cụ **Database Version Control** (quản lý phiên bản mã nguồn cho cơ sở dữ liệu), đóng vai trò tương tự như Git đối với mã nguồn Java/TypeScript.
- **3 Chức năng cốt lõi:**
 1. **Tự động hóa:** Khi Spring Boot khởi động, Flyway tự động rà soát và thực thi tuần tự các file script SQL mà không cần can thiệp thủ công.
 2. **Đồng bộ môi trường:** Đảm bảo toàn bộ 6 thành viên trong nhóm và máy chủ production đều sở hữu cùng một cấu trúc database duy nhất.
 3. **Kiểm toán tính toàn vẹn (Checksum):** Ngăn chặn việc sửa lén script cũ thông qua cơ chế so sánh mã băm.

2. Sơ đồ luồng vận hành Flyway Migration



3. Cách Show Code Flyway Cho Thầy Cô Xem

- **Mở thư mục Migration:** Đường dẫn `backend/src/main/resources/db/migration/` (gồm 9 file từ **V001** → **V009**).
- **Quy ước đặt tên bắt buộc:** `V<Phiên_bản>__<Mô_tả>.sql` (bắt buộc phải có 2 dấu gạch dưới `__`).
- **Cấu hình Spring Boot:** File `backend/src/main/resources/application.yml`:

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate # Hibernate chỉ đối chiếu Entity, cấm tự ý sửa bảng
  flyway:
    enabled: true # Bật Flyway tự động chạy
    locations: classpath:db/migration
```

- **Đối chiếu trên pgAdmin 4:** Chạy câu lệnh truy vấn bảng `flyway_schema_history` :

```
SELECT installed_rank, version, description, script, checksum,
       installed_on AT TIME ZONE 'Asia/Ho_Chi_Minh' AS thoi_gian_chay, success
FROM flyway_schema_history ORDER BY installed_rank ASC;
```

4. Bộ Câu Hỏi Bẫy Của Thầy Cô Về Flyway & Câu Trả Lời A+

Hỏi: Tại sao không dùng `spring.jpa.hibernate.ddl-auto=update` cho nhanh mà phải dùng Flyway?

Đáp: Dùng `ddl-auto=update` của Hibernate rất nguy hiểm trên Production vì Hibernate không thể xóa cột cũ an toàn, dễ gây mất mát dữ liệu và không lưu lại lịch sử ai đã sửa/bảng nào. Do đó nhóm dùng `ddl-auto=validate` (Hibernate chỉ kiểm tra tính hợp lệ) và giao toàn quyền tiến hóa schema cho Flyway.

Hỏi: Nếu em lén sửa nội dung file `V002__seed_data.sql` đã chạy rồi thì chuyện gì sẽ xảy ra?

Đáp: Khi sửa nội dung, mã băm (checksum) của file sẽ bị đổi. Ở lần khởi động tiếp theo, Flyway tính lại checksum và so với giá trị trong bảng `flyway_schema_history`, thấy không khớp sẽ lập tức ném lỗi `FlywayValidateException` và **từ chối khởi động hệ thống** để bảo vệ dữ liệu.

PHẦN 4: GIAO TIẾP DỮ LIỆU & KIẾN TRÚC API

1. API là gì? Lấy API ở đâu?

- **API (Application Programming Interface):** Là cổng giao tiếp dữ liệu có cấu trúc giữa Frontend và Backend. Thay vì trả về giao diện HTML, Backend cung cấp các API trả về dữ liệu chuẩn JSON qua các phương thức HTTP: `GET` (truy vấn), `POST` (tạo mới), `PUT/PATCH` (cập nhật), `DELETE` (xóa).
- **Lấy API ở đâu trong dự án SEAL Hackathon?**
 1. **Tài liệu trực quan Swagger UI:** Mở trình duyệt tại địa chỉ `http://localhost:8080/swagger-ui.html` khi backend đang chạy. Thầy cô có thể test trực tiếp từng API trên trình duyệt.
 2. **Đặc tả OpenAPI JSON:** Đường dẫn `http://localhost:8080/v3/api-docs`.
 3. **Mã nguồn Backend Controller:** Thư mục `backend/src/main/java/com/seal/hackathon/controller/` (gồm 15 Controllers: `AuthController`, `TeamController`, `SubmissionController`, `ScoreController`, `RankingController` ...).
 4. **Mã nguồn Frontend API Client:** Thư mục `frontend/src/api/` (các module TypeScript: `events.ts`, `teams.ts`, `submissions.ts`, `scores.ts`, `rankings.ts` ...).

2. Kiến Trúc 3 Tầng Bảo Mật BFF (Backend-For-Frontend)

Tầng Kiến Trúc	Công Nghệ Sử Dụng	Trách Nhiệm & Vai Trò Cốt Lõi
1. Client Frontend	React 18, TypeScript, TailwindCSS, Vite (Cổng 3000)	Giao diện người dùng tương tác, render bảng điểm dạ quang, xử lý form nộp bài, đọc cookie CSRF gắn vào request header.
2. Gateway BFF	NestJS, Fastify/Express (Cổng 4000)	Cổng kiểm soát an ninh: Quản lý Cookie <code>httpOnly</code> cho JWT, phát hành và kiểm tra token CSRF hai lớp, chống tấn công trực diện vào Core Backend.
3. Core Backend	Spring Boot 3, Java 17, Spring Security (Cổng 8080)	Xử lý toàn bộ nghiệp vụ lõi (Business Logic): Thuật toán tính điểm Rubric, phân tích phương sai RBL, tính xếp hạng Realtime và ghi nhật ký Audit Log.
4. Database Layer	PostgreSQL 16, Flyway Migration (Cổng 5433)	Lưu trữ dữ liệu quan hệ bền vững, quản lý phiên bản schema tự động.

PHẦN 5: BẢO MẬT HỆ THỐNG — PHÒNG THỦ CSRF & XSS

1. CSRF là gì? (Có mở ngoặc chú thích rõ ràng)

CSRF viết tắt của **Cross-Site Request Forgery** (*Tấn công giả mạo yêu cầu từ trang web khác*). Đây là hình thức tấn công mà kẻ xấu lợi dụng đặc tính tự động gửi kèm cookie phiên của trình duyệt để thực hiện các thao tác phá hoại mà nạn nhân không hề hay biết.

2. Tại sao SEAL Hackathon lại có nguy cơ bị CSRF?

- Để triệt tiêu hoàn toàn nguy cơ **XSS** (*Cross-Site Scripting* — *Tấn công tiêm mã độc vào trình duyệt*), nhóm quyết định **không lưu Access Token trong `localStorage`**.
- Toàn bộ token được bảo vệ trong Cookie `httpOnly` (JavaScript bị cấm truy cập). Tuy nhiên, dùng Cookie thì trình duyệt lại tự động gửi kèm mỗi request, dẫn đến nguy cơ bị tấn công **CSRF**.

3. Cơ Chế Phòng Thủ Double-Submit CSRF Cookie Tại Tầng BFF

1. **Cấp phát Token:** Khi người dùng đăng nhập, BFF tạo một chuỗi ngẫu nhiên bí mật và trả về qua cookie có tên là `XSRF-TOKEN` (cho phép JavaScript đọc).
2. **Gửi kèm Header:** Khi Frontend (React) thực hiện các lệnh làm thay đổi dữ liệu (`POST` , `PUT` , `DELETE`), mã nguồn tại `frontend/src/api/http.ts` sẽ đọc cookie `XSRF-TOKEN` và tự động gắn vào Header: `X-XSRF-TOKEN`.
3. **Kiểm soát tại Guard:** Bộ lọc `CsrfGuard.ts` tại BFF đối soát:

Cookie `[XSRF-TOKEN]` === Header `[X-XSRF-TOKEN]` ?

Nếu trùng khớp → Cho phép chuyển tiếp xuống Spring Boot. Nếu thiếu hoặc sai lệch → Lập tức chặn với mã lỗi **HTTP 403 Forbidden**.

4. **Nguyên lý bảo vệ tuyệt đối:** Dựa trên chính sách **SOP (Same-Origin Policy)** của trình duyệt, trang web độc hại từ bên ngoài **không thể nào đọc được cookie của SEAL Hackathon**, vì vậy kẻ tấn công không thể giả mạo được Header `X-XSRF-TOKEN`. Cuộc tấn công bị ngăn chặn 100%!

PHẦN 6: KỊCH BẢN DEMO THỰC TẾ & ĐỐI CHIẾU PGADMIN 4

1. Checklist 9 Bước Demo Toàn Diện Hệ Thống

Bước	Thao tác trên Giao diện Web	Tài khoản sử dụng	Kỳ vọng Giao diện	Kỳ vọng trong PostgreSQL
1	Đăng ký tài khoản thí sinh & Ban tổ chức duyệt	svfpt@gmail.com → coordinator@demo.local	Thông báo đã duyệt thành công	account_status = 'APPROVED'
2	Tạo đội thi mới	svfpt@gmail.com	Đội AI CHAMPIONS xuất hiện	Bảng team có status = 'FORMING'
3	Mời thành viên vào đội	Đội trưởng (mtai@gmail.com)	Gửi lời mời thành công	Bảng team_invite có PENDING
4	Thành viên chấp nhận lời mời	minhtai@gmail.com	Thấy tên trong danh sách đội	Bảng team_member thêm bản ghi
5	Đăng ký Hạng mục thi đấu	Đội trưởng (≥ 3 thành viên)	Chọn Mobile Application	team.status = 'REGISTERED'
6	Nộp bài dự thi	Đội trưởng	Điền link GitHub, Demo, Slide	Bảng submission có is_late = true
7	Chấm điểm Hiệu chuẩn (RBL)	Giám khảo judge1@demo.local	Kéo slider chấm bài mẫu	Lưu vào bảng calibration_score
8	Chấm điểm thi chính thức	Giám khảo judge1@demo.local	Chấm 4 tiêu chí, bấm Chốt điểm	Lưu vào bảng score (finalized = t)
9	Kiểm tra Xếp hạng Realtime	Khách vãng lai / Toàn trường	Mở trang /rankings	Bảng ranking tự động nhảy số Top 1

2. Tập Lệnh SQL Đối Chiếu Trực Tiếp Trên pgAdmin 4

```
-- 1. Xem bài nộp và thời gian nộp bài quy đổi giờ Việt Nam
SELECT t.name AS ten_doi, s.repo_url, s.demo_url,
       s.submitted_at AT TIME ZONE 'Asia/Ho_Chi_Minh' AS gio_nop_viet_nam, s.is_late AS nop_muon
FROM submission s JOIN team t ON s.team_id = t.id ORDER BY s.submitted_at DESC;

-- 2. Xem điểm hiệu chuẩn của các giám khảo (RBL Calibration)
SELECT cr.name AS phien_hieu_chuan, u.full_name AS giam_khao, c.name AS tieu_chi, cs.score_value AS diem
FROM calibration_score cs
JOIN calibration_round cr ON cs.calibration_round_id = cr.id
JOIN app_user u ON cs.judge_id = u.id
JOIN criterion c ON cs.criterion_id = c.id;

-- 3. Xem bảng xếp hạng chính thức sau khi giám khảo chấm điểm
SELECT rk.rank_overall AS hang, t.name AS ten_doi, tr.name AS hang_muc,
       rk.total_weighted_score AS diem_tong_hop, rk.promoted AS vao_chung_ket
FROM ranking rk
JOIN team t ON rk.team_id = t.id
LEFT JOIN track tr ON t.track_id = tr.id ORDER BY rk.rank_overall ASC;

-- 4. Soi vết kiểm toán bất biến (Audit Log)
SELECT id, action, entity_type, timestamp AT TIME ZONE 'Asia/Ho_Chi_Minh' AS thoi_gian
FROM audit_log ORDER BY timestamp DESC LIMIT 10;
```

PHẦN 7: BỘ CÂU HỎI TRỌNG YẾU HỘI ĐỒNG CHẮC CHẮN SẼ HỎI

Câu 1: Điểm hiệu chuẩn (Calibration Score) và Điểm thi thật (Score) khác nhau thế nào? Tại sao phải tách thành 2 bảng?

Trả lời: Điểm hiệu chuẩn là điểm giám khảo chấm thử trên bài mẫu (Sample Submission) trước giờ thi để làm quen thước đo và phục vụ nghiên cứu khoa học (đo hệ số tin cậy liên đánh giá viên IRR). Việc tách thành 2 bảng `calibration_score` và `score` đảm bảo các dữ liệu nghiên cứu thử nghiệm không bao giờ làm xáo trộn điểm thi và bảng xếp hạng chính thức của thí sinh.

Câu 2: Tại sao khi nộp bài cờ `is_late` lại bằng `true (t)` và hệ thống xử lý bài nộp trễ như thế nào?

Trả lời: Trong cơ sở dữ liệu mẫu, hạn chót nộp bài của Vòng Chung Kết là ngày `20/09/2026 23:59:59`, trong khi thời điểm nộp bài thực tế là ngày `23/09/2026`. Thuật toán tại `SubmissionService.java` so sánh `submittedAt.isAfter(deadline)` thấy trễ nên tự động đánh dấu `is_late = true`. Khi tính điểm xếp hạng tại `RankingService.java`, bài nộp trễ sẽ tự động bị phạt trừ 10% tổng điểm có trọng số nhằm đảm bảo công bằng.

Câu 3: Làm thế nào để Bảng xếp hạng cập nhật ngay lập tức (Realtime) khi Giám khảo vừa bấm Lưu điểm?

Trả lời: Trước đây việc tính điểm phải chờ Ban Tổ Chức bấm nút thủ công. Nhóm đã tối ưu trong `ScoreService.java`: Ngay khi Giám khảo bấm "Lưu" hoặc "Chốt điểm", hệ thống sẽ phát sinh sự kiện nội bộ và tự động gọi thuật toán `rankingService.compute()` chạy ngầm ngay tức khắc. Nhờ đó bảng xếp hạng tại `/rankings` luôn hiển thị kết quả chính xác theo thời gian thực 100%.

Câu 4: Bảng `audit_log` bảo vệ hệ thống trước sự cố gian lận như thế nào?

Trả lời: Mọi thao tác thay đổi trạng thái (Duyệt user, Nộp bài, Sửa điểm, Khóa điểm, Loại đội thi) đều tự động ghi lại Actor, Action, Entity, Timestamp và IP vào bảng `audit_log`. Hệ thống không cung cấp bất kỳ API nào để cập nhật hay xóa bảng này (Append-only trail), đảm bảo tính minh bạch, khách quan và chống chối bỏ trách nhiệm.